Exercise 5: Task Management System

1. Aim

To develop a Task Management System using a **Singly Linked List** in Java and implement operations such as adding, searching, traversing, and deleting tasks.

2. Problem Understanding

A Task Management System stores and manages tasks such as task creation, searching, updating, and deletion.

Unlike arrays, linked lists do not require contiguous memory allocation. Each node stores the task information and a reference to the next node.

Linked lists are suitable when the number of records changes frequently because they allow dynamic memory allocation

3. Types of Linked Lists

Singly Linked List

Each node contains:

- Data
- Reference to the next node

Head
↓
[Task1] → [Task2] → [Task3] → NULL

Doubly Linked List

Each node contains:

- Previous reference
- Data
- Next reference

NULL ← [Task1] ⇄ [Task2] ⇄ [Task3] → NULL

4. Algorithm

Add Task

1. Create a new node.
2. If the list is empty, make it the head.
3. Otherwise, traverse to the last node.
4. Insert the new node.

Search Task

1. Start from the head node.
2. Compare the task ID.
3. If found, display the task.

Traverse Tasks

1. Start from the head.
2. Visit every node.
3. Display task details.

Delete Task

1. Search for the task.
2. Update the previous node's next reference.
3. Remove the node.

5. Program\Output:

Exercise5.java > Exercise5 > main(String[])

```
class Task{
    int taskId;
    String taskName;
    String status;

    Task next;

    public Task(int taskId,String taskName,String status){
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
    }
    @Override
    public String toString(){
        return "Task ID: "+taskId+", Task Name: "+taskName+", Status: "+status;
    }
}
public class Exercise5 {
    static Task head = null;
    public static void addTask(Task newTask){
        if(head == null){
            head = newTask;
            return;
        }

        Task temp = head;

        while(temp.next!=null){
            temp = temp.next;
        }
        temp.next = newTask;
    }
    public static void searchTask(int id){
        Task temp= head;
        while(temp!=null){
            if(temp.taskId == id){
                System.out.println(temp);
                return;
            }
            temp = temp.next;
        }
        System.out.println(x: "Task Not Found.");
    }
    public static void displayTasks(){
        Task temp = head;
        System.out.println(x: "\nTask List:");
        while(temp!=null){
            System.out.println(temp);
            temp = temp.next;
        }
    }
```

```java
    public static void deleteTask(int id){
        if(head == null){
            return;
        }
        if(head.taskId == id){
            head = head.next;
            System.out.println(x: "Task Deleted Successfully.");
            return;
        }
        Task current = head;
        Task previous = null;
        while(current!=null && current.taskId!=id){
            previous = current;
            current = current.next;
        }
        if(current!=null && current.taskId!=id){
            System.out.println(x: "Task Not Found.");
            return;
        }
        previous.next = current.next;
        System.out.println(x: "Task Deleted Successfully.");
    }
    Run | Debug
    public static void main(String args[]){
        addTask(new Task(taskId: 101,taskName: "Design UI", status: "pending"));
        addTask(new Task(taskId: 102,taskName: "Develop Backend", status: "In Progress"));
        addTask(new Task(taskId: 103,taskName: "Testing", status: "pending"));

        displayTasks();

        System.out.println(x: "\nSearching Task:");
        searchTask(id: 102);

        System.out.println();

        deleteTask(id: 102);
        displayTasks();
    }
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Task List:
Task ID: 101, Task Name: Design UI, Status: pending
Task ID: 102, Task Name: Develop Backend, Status: In Progress
Task ID: 103, Task Name: Testing, Status: pending

Searching Task:
Task ID: 102, Task Name: Develop Backend, Status: In Progress

Task Deleted Successfully.

Task List:
Task ID: 101, Task Name: Design UI, Status: pending
Task ID: 103, Task Name: Testing, Status: pending
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\Algorithm>
```

6. Time Complexity Analysis

Operation	Time Complexity
Add Task	O(n)
Search Task	O(n)
Traverse Tasks	O(n)
Delete Task	O(n)

7. Result

Thus, the **Task Management System** was successfully implemented using a **Singly Linked List**. The system supports adding, searching, traversing, and deleting tasks efficiently. Linked lists provide dynamic memory allocation and simplify insertion and deletion operations, making them more suitable than arrays for applications with frequently changing data.